

2DX3 Final Report

Tony Tran

400572083

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is our own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario.

Contents

1/ Device Overview – Features.....	1
2/ General Description	2
Block Diagram	2
Characteristic Table:.....	3
3/ Detailed Description.....	4
3.1/ Distance Measurement Acquisition:	4
3.2/ Distance Measurement:	4
3.3/ Visualization:	5
4/ Application:	5
4.1/ Example	5
4.2/ Instructions:	5
4.3/ XYZ Coordinate.....	6
4.4/Expected Output:.....	6
5/ Limitations:.....	8
6/ Maximum Standard Serial Communications.....	8
7/ Communication Methods and Speed	8
8/ Primary Limitation.....	9
9/ Circuit Schematic	9
10/ Flowcharts	11

1/ Device Overview – Features

MSP432E401Y SimpleLink™ Ethernet Microcontroller

- **Bus speed:** up to 120MHz
- **Operating Voltage:** 3.3-5V
- **Cost:** \$200
- **I2C Interface:** up to 3.3 MHz
- **Baud Rate:**
 - **UART:** up to 128 kbps
- **Programming Language for Visualization:** MATLAB
- **Memory Information:** 1024 Kb of flash memory and 256 Kb SRAM

VL53L1X ToF Sensor

- **Ranging Frequency:** up to 50Hz
- **Max Ranging Distance:** up to 400cm
- **I2C Interface:** Up to 400 KHz
- **Operating Voltage:** 2.6-3.5V
- **Cost:** Included when buying the microcontroller
- **Memory Information:** Non-volatile memory

VMA401 Stepper Motor

- **Frequency:** up to 100Hz
- **Number of phases:** 4
- **Steps/revolution:** 64
- **Operating Voltage:** 5-12V
- **Cost:** Included when buying the microcontroller
- **Stride Angle:** 5.625 degrees/64

Programming Language: MATLAB

- **Used Library:** colour map, plot, and scatter functions
- **Cost:** Free
- **Usage:** Plot a 3D plot of the collected data

Communication Protocols:

- **I2C:** Communication between ToF and the microcontroller
- **UART:** Communication between the microcontroller and the PC with a baud rate of 115200 bps.

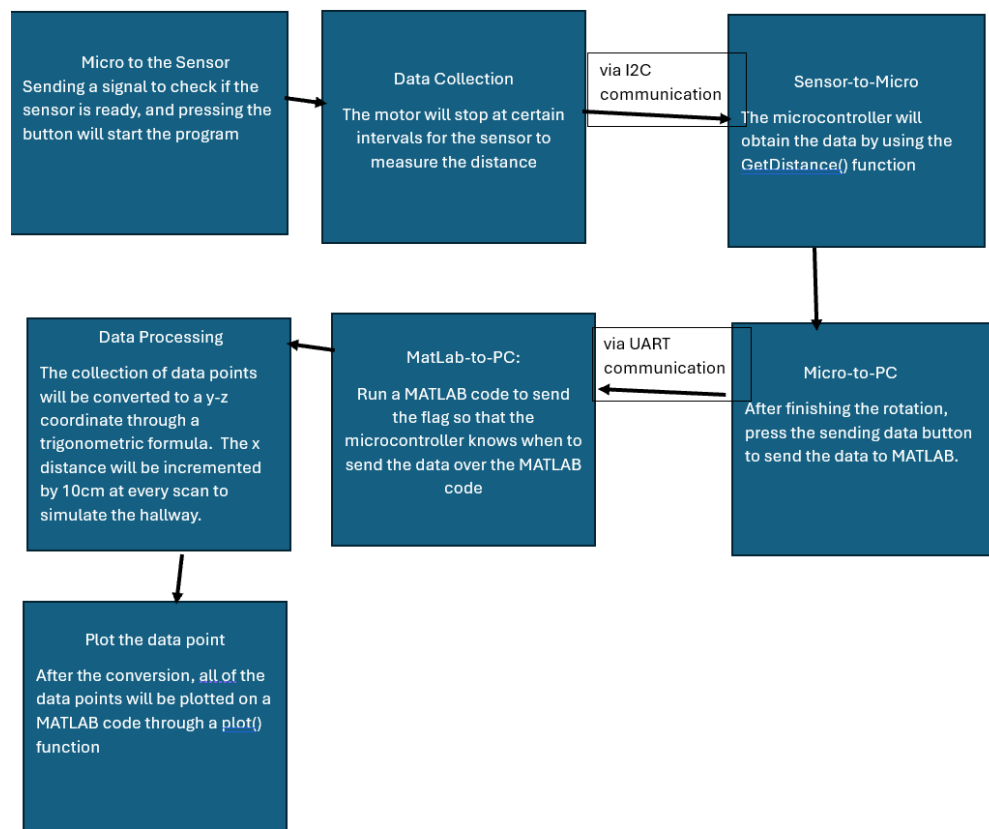
2/ General Description

In the MSP432E401Y, signal acquisition is the process of collecting analog signals from sensors through analog input pins. These signals will undergo preprocessing such as filtering, amplification, or conditioning to ensure accuracy and compatibility with the ADC input range. The microcontroller will then use its built-in 12-bit ADC conversion to convert the analog voltage to a digital value, which can be used for transmitting or analyzing in the system.

UART is a serial communication used to transmit data from the microcontroller to the PC and then display it on Realtem. UART operates by sending each bit at a time and uses the `UART_printf()` function to transmit the data to the PC.

The visualization of data is conducted through MatLab that involves the use of trigonometric formulas to convert the obtained data to a y-z coordinate. Y will be a vertical line, whereas z will be a horizontal line on the y-z plane. Hence, the horizontal distance will be $z = \text{distance} * \sin(\theta)$, whereas the vertical distance will be $y = \text{distance} * \cos(\theta)$.

Block Diagram



Characteristic Table:

Table 1:Characteristic Table

MSP432E401Y SimpleLink™ Ethernet Microcontroller	Features	Sensor	Pin Used	Stepper Motor	Pin Used
Bus Speed	20MHz	SCL	PB2	IN1	PH0
COM Port	COM4	SDA	PB3	IN2	PH1
Communication Protocols	I2C and UART	Vin	3.3V	IN3	PH2
Baud Rate	115200Bps	GND	GND	IN4	PH3
Measurement, Status, and Additional LED	PF4, PF0, PN0	-	-	V+	5V
-	-	-	-	V-	GND
-	-	-	-	-	

3/ Detailed Description

3.1/ Distance Measurement Acquisition:

The time-of-flight sensor (ToF) works by emitting a pulse of light of 940nm. The sensor will then calculate the time it takes for emitted light pulses to reach the nearest point and be reflected to the detector. The distance will be calculated by timing the speed of light with the photon travel time divided by 2. After the data is collected, it will undergo a process of transducing, signal conditioning, and converting it from analog to digital values. It is noticeable that this process is already included in the sensor, as it provides a digital output.

Following the data collection, the microcontroller needs to collect the data from the sensor. This stage requires I2C communication that involves the use of API functions. Before the stepper motor rotates, the microcontroller will check if the sensor is booted through the VL53L1X_BootState(dev, &sensorState) function. If the device is booted, pressing PJ0 will make the motor rotate and start the measurement. The measurement is obtained by using the function getDistance(dev, &Distance). Once the measurement is completed, VL53L1X_ClearInterrupt(dev) will be executed to clear the interrupt so that the next interrupt can be enabled.

In addition to I2C communication, the number of measurements is 32 data points, as the motor will stop to measure the distance at every interval of 11.25 degrees. In Keil, there will be a conditional statement that checks if it loops 16 times; it will start collecting data. The reason for this is that one iteration will run four steps, so 11.25 degrees will require 16 steps, as it takes the motor 512 iterations to finish one full rotation.

$$\text{Measured Distance} = \frac{V_{\text{light}} * t_{\text{photon}}}{2}$$

$$\text{Number of required step for 11.25 degrees} = \frac{11.25 * 512}{360} =$$

16 steps

3.2/ Distance Measurement:

After obtaining the data from GetDistance(), PJ1 has to be pressed to enter the transmitting data stage. The Keil code will wait for the flag from MATLAB, so MATLAB has to run before the data can be sent. Following this stage, the data will be printed and entered into the conversion stage. To convert the collected data into y-z coordinates, two matrices will be created for each y and z coordinate of the obtained distance. The formulas to calculate y and z are:

$$y = \text{Distance} * \sin(\theta) \text{ and } z = \text{Distance} * \cos(\theta)$$

For instance, if the obtained distance is 15mm and theta is 90 degrees, then y will be 15mm, and z will be zero. Notably, my implemented design assumed the reference degree would be 90 degrees, and the angle would be incremented by certain angles based on the design choice. In this case, it is 11.25 degrees. To track the change in the angle of each data point, another matrix is formed to store the angle of each data point. The number of angles will be predefined based on how many data points users want to collect.

Based on the project requirements, the x-displacement will remain constant and will be incremented by 10 cm for each scan. However, when scanning the object, it is suggested that users should move 30cm each scan to produce a high-quality scan. To implement the displacement in MATLAB, a matrix of x-coordinates will be initialized and put in the for loop that iterates over the number of scans. The x-coordinate will be incremented by 10 each time based on the number of scans.

3.3/ Visualization:

The program used to produce a 3-D scan of the hallway is MATLAB. It uses the scatter3() function to display the data on the graph, while the plot3() function will connect data points within each scan. Notably, there is a vector concatenation that appends the first point to the end of the coordinate arrays inside the plotting loop before executing the plot3() function. This function ensures that lines connecting the last point to the starting point are drawn, forming a closed loop that better represents the spatial structure of the hallway. To distinguish the difference in each scanning area, the jet() colour map function is used to change the colour for every scan. The graph is further improved by using the axis equal function to maintain consistent scaling, the grid on function to provide spatial references, and the axis labels function to indicate the coordinate system, making it easier for readability and analysis of the 3D map.

4/ Application:

4.1/ Example

The design is a Light Detection and Ranging (LIDAR) equipment that uses pulse lasers to collect data of the scanning area for a 3D graph. This equipment is widely used in mining industries where mining areas will be mapped to a 3D graph to collect essential information about the geological data. This ensures that every dangerous area can be detected to provide safety for miners. Moreover, LIDAR is also very handy in autonomous vehicles and robotics as it provides a 3D map that can be used for object detection and navigation.

4.2/ Instructions:

To ensure that the device works as expected, users need to follow these steps:

1. Plug the microcontroller into a computer and determine its COM port. The COM port can be found by using 'Device Manager', looking under Ports (COM & LPT), and finding XDS110 User UART. The number at the end of the XDS110 User UART will be a port number.

2. Open the MATLAB script. Then, the port variable needs to be changed to the port number found in step 1, and the baud rate needs to be adjusted to 115200 bps.
3. Configure all the pins as shown in the characteristic table.
4. Open Keil Project. Find 'Options for Target', then 'C/C++ tab', and check if the optimization is '-O0'. Following this, go to the 'Debug' tab and select 'CMSIS-DAP Debugger' in the top right drop-down menu. After setting the CMSIS-DAP Debugger, click 'Settings' and select 'XDS110 with CMSIS-DAP' from the drop-down menu. Next, go to the 'Target' Tab and ensure that the 'Use default compiler version 6' option is picked for the ARM Compiler. Click OK to close the 'Options for Target' tab.
5. Click 'Translate', 'Build', and 'Load' in this order.
6. Click the reset button on the microcontroller to flash the project onto the board.
7. Open Keil. Change the number of data points you want to measure
8. Open MATLAB. Do the same thing as step 7.
9. Press PJ0 to begin the measurement
10. Once you get enough scans, press PJ1 to begin the data transfer.
11. Run the code in MATLAB. Press Enter to see the data points and the 3D of your scan

4.3/ XYZ Coordinate

The coordinate of the XYZ is defined as yz for the vertical distance slice, and x for displacement.

4.4/Expected Output:

The last digit of my student number is 3, so I am assigned to scan the hallway at location D on the second floor of ITB. My 3D plot outlines all the properties of the hallway, shown in different colours. In the blue plot, points that extend all the way to the left are the empty hallway. This is considered the anomaly of this hallway because there is no wall for the light to reflect to the sensor; hence, I set the data point as 4000mm when the range status is 2. Another property is the washroom door on the left-hand side of the hallway. The yellow graph illustrates the 231A door, and the brown one illustrates the 230A door. This is because the data points of these two graphs extend more on the left compared to the red, orange, blue, and green scans, which all scan at the middle of the hallway without any doors on the left-hand side. It is noticeable that those two doors on the left-hand side of the given diagram extend behind the wall, whereas those on the right-hand side are on the same side as the wall. That explains why the graph extends more on the left-hand side when scanning the door.

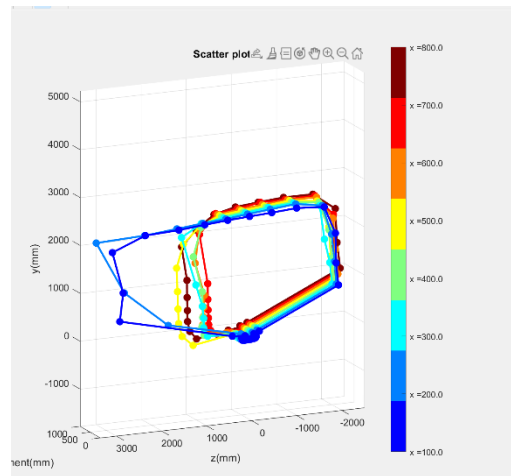


Figure 1: Front view of the hallway

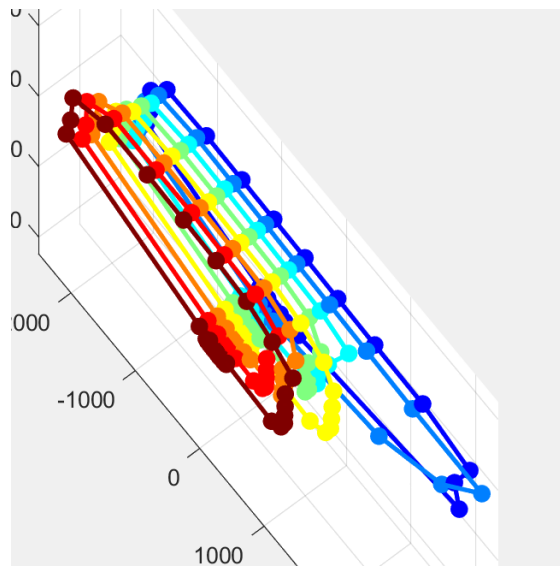


Figure 2: Top view of the hallway

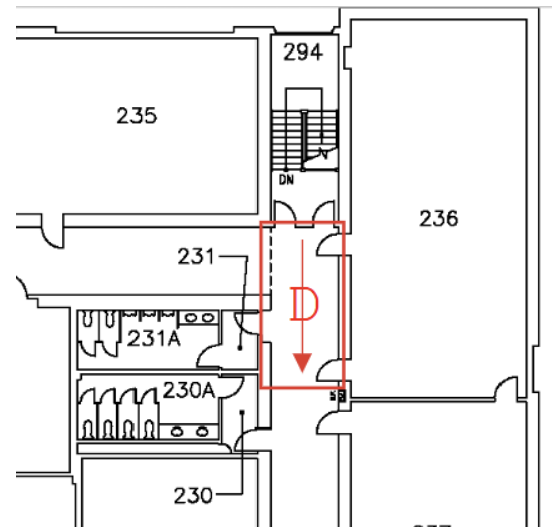


Figure 3: The assigned location for scanning

5/ Limitations:

One limitation of the MSP432E401Y is the use of an IEEE754-compliant single-precision Floating-Point Unit (FPU) that allows the microcontroller to calculate decimal numbers. However, it is still limited in accuracy compared to a full computer system because it is only 32-bit compared to 64-bit on a PC. This will reduce numerical accuracy and slow computational time with complex calculations. The precision will become even less accurate when using trigonometric functions, such as sine and cosine. This is because the conversion of the distance data point to Cartesian coordinates will only be an approximate value stored in MATLAB. Hence, combining these two limitations will introduce more percentage error for the output value mapped on the 3D-plot.

6/ Maximum Standard Serial Communications

The maximum standard serial communication rate that I can implement with the PC is 128000 bps. This can be found by going to 'Device Manager', clicking Ports (COM & LPT). Then, right-click the 'User UART (COM4)' and choose 'Properties'. In the 'Properties' tab, go to port settings, click on the drop-down menu of 'Bits per second', and scroll down to find the maximum serial communication rate.

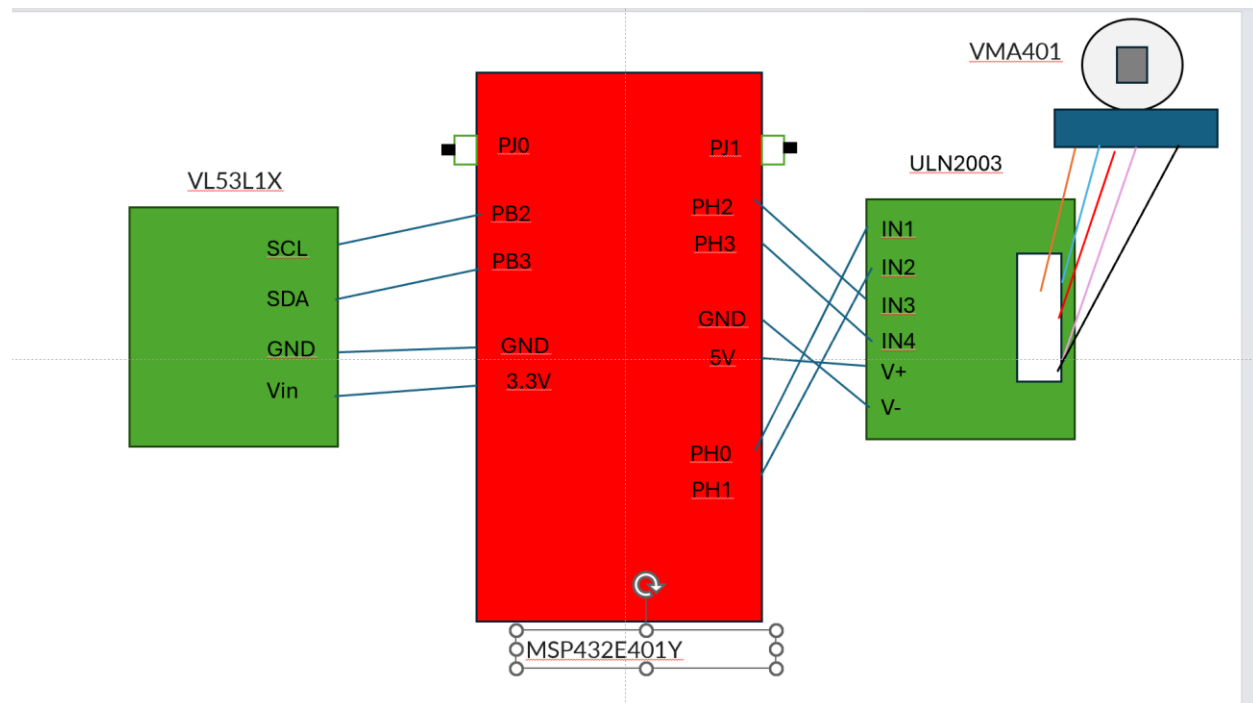
7/ Communication Methods and Speed

The communication which is used between the microcontroller and ToF is I2C communication that transmits data through SDA, whereas the SCL line will be for the clock signal for data synchronization. The speed used between the microcontroller and the ToF is up to 400 kHz for fast mode.

8/ Primary Limitation

The primary limitation of the design is the delay time between each step of a stepper motor. This is because the total delay time for ToF is only 0.17s, whereas the total delay time for the stepper motor to finish one rotation and rotate back to unwire is around 16.384s. The total delay time of ToF is from 0.01s to wait for device boot, and a total of 0.16s to wait for the data to be ready, calculated when the motor rotates every 11.25 degrees, which is 32 times. On the other hand, the motor will require a delay of 4ms per step, which runs 512 times to rotate one full rotation, and it will rotate backward another 512 times to untangle the wire. This leads to a total higher delay of 16.384s. To test the primary bottleneck, different delay values were tested. When the delay time is 8ms, the motor barely moves. I then decrement it to 5ms, which makes it rotate, but the rotation is very slow. The delay time of 4ms is ideal, as the stepper motor rotates smoothly and consistently. This testing indicates the primary limitation is the delay time from the stepper motor.

9/ Circuit Schematic



10/ Flowcharts

